DATABASE DESIGN

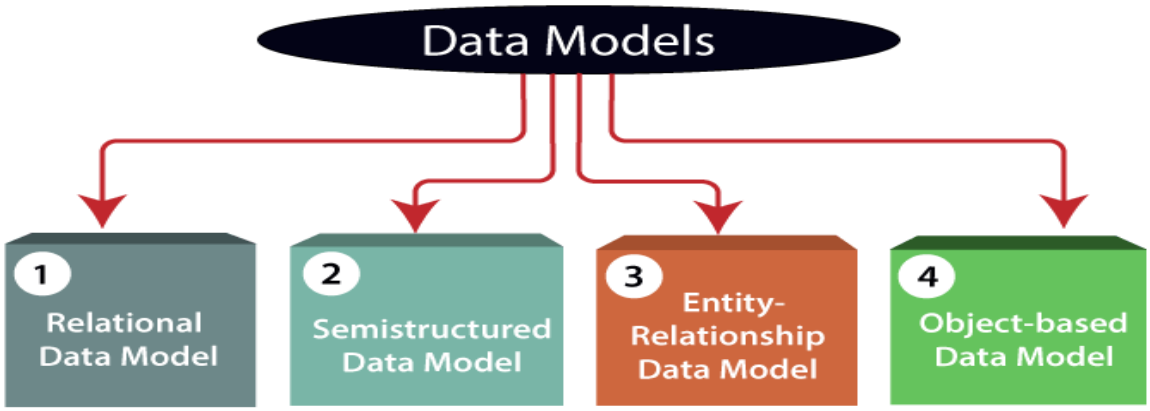
Introduction

2. 1 Data Model:

A data model is used in the database design process. It aims to identify & organize the required data. A data model says what information is to be contained in the database, how the information will be used, and how the items in the database will be related to each other.

The data model is used to describe the structure of the data base- data types, relationships & constraints that should hold for the data. Data model include a set of operations for specifying retrievals & updates on the data base.

Types of data Models:



1) Relational Data Model: This type of model designs the data in the form of rows and columns within a table. Thus, a relational model uses tables for representing data and in-between relationships. Tables are also called relations. This model was initially described by Edgar F. Codd, in 1969. The relational data model is the widely used model which is primarily used by commercial data processing applications.

2) Entity-Relationship Data Model: An ER model is the logical representation of data as objects and relationships among them. These objects are known as entities, and relationship is an association among these entities. This model was designed by Peter Chen and published in 1976 papers. It was widely used in database designing. A set of attributes describe the entities. For example, student_name, student_id describes the 'student' entity. A set of the same type of entities is known as an 'Entity set', and the set of the same type of relationships is known as 'relationship set'.

3) Object-based Data Model: An extension of the ER model with notions of functions, encapsulation, and object identity, as well. This model supports a rich type system that includes structured and collection types. Thus, in 1980s, various database systems following the object-oriented approach were developed. Here, the objects are nothing but the data carrying its properties.

4) **Semistructured Data Model:** This type of data model is different from the other three data models (explained above). The semistructured data model allows the data specifications at places where the individual data items of the same type may have different attributes sets. The Extensible Markup Language, also known as XML, is widely used for representing the semistructured data. Although XML was initially designed for including the markup information to the text document, it gains importance because of its application in the exchange of data.

2.1.1 Importance Of Data Modelling :

A data model helps design the data base at the conceptual, physical & logical levels. Data model structure helps to define the relational tables, primary and foreign keys and stored procedures. It provides a clear picture of the base data and can be used by data base developers to create a physical database.

2.2 Database Design

Phases of Database Design:

The database design using high-level conceptual data models consists of different phases as shown in the figure.2.1.

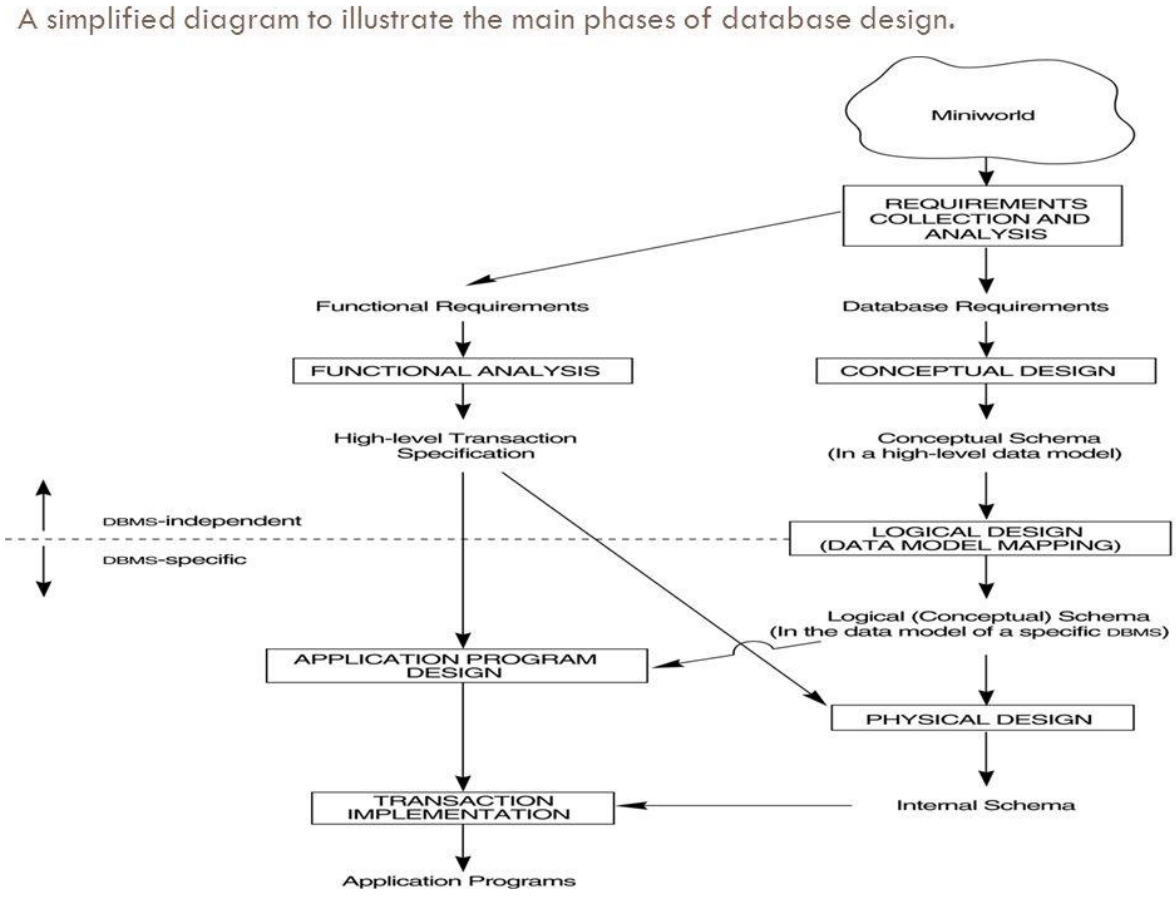


Figure 2.1 a simplified overview of the database design process

Requirements collection and analysis

The database designers interview database users to understand and document their data requirements. These requirements should be specified in as detailed and complete.

In parallel with specifying the data requirements, it is useful to specify the functional requirements of the users.

Functional requirements consist of the user defined operations that will be applied to the database, including both retrieval and updates.

Conceptual design

Once all the requirements have been collected and analyzed, the next step is to create a conceptual schema for the database using high level conceptual data model. This step is called conceptual

Conceptual schema includes detailed description of the entity types, relationships and constraints.

Logical design

After the conceptual design, the next step in database design is the actual implementation of the database, using a commercial DBMS.

The conceptual schema is transformed from the high-level data model into the implementation data model using DBMS.

This step is called **logical design** or **data model mapping**; its result is a database schema in the implementation data model of the DBMS.

Physical Design

The last step is the physical design phase, during which the internal storage structures, file organizations, indexes, access paths, and physical design parameters for the database files are specified.

In parallel with these activities, application programs are designed and implemented as database transactions.

2.3 ER-Model

ER Model is used to model the conceptual view of the system from data perspective.

which consists of these components:

1. Entities

2. Attributes
3. Relationships

The ER model describes data as *entities, relationships*, and *attributes*.

ER diagrams: The schema for database application can be displayed by means of the graphical notation known as **ER diagrams.**

Entity, Entity Type, Entity Set –

The basic object that the ER model represents is an **entity**, which is a thing in the real world with an independent existence. An entity may be an object with a physical existence (for example, EMPLOYEE entity, STUDENT entity and etc).

An **entity type** defines a collection (or set) of entities that have the same attributes. Each entity type in the database is described by its name and attributes.

Example: EMPLOYEE Entity type with Name, age, salary so on attributes.

The collection of all entities of a particular entity type in the database at any point in time is called an **entity set**; the entity set is usually referred to using the same name as the entity type.

For example, EMPLOYEE refers to both a type of entity as well as the current set of all employee entities in the database.

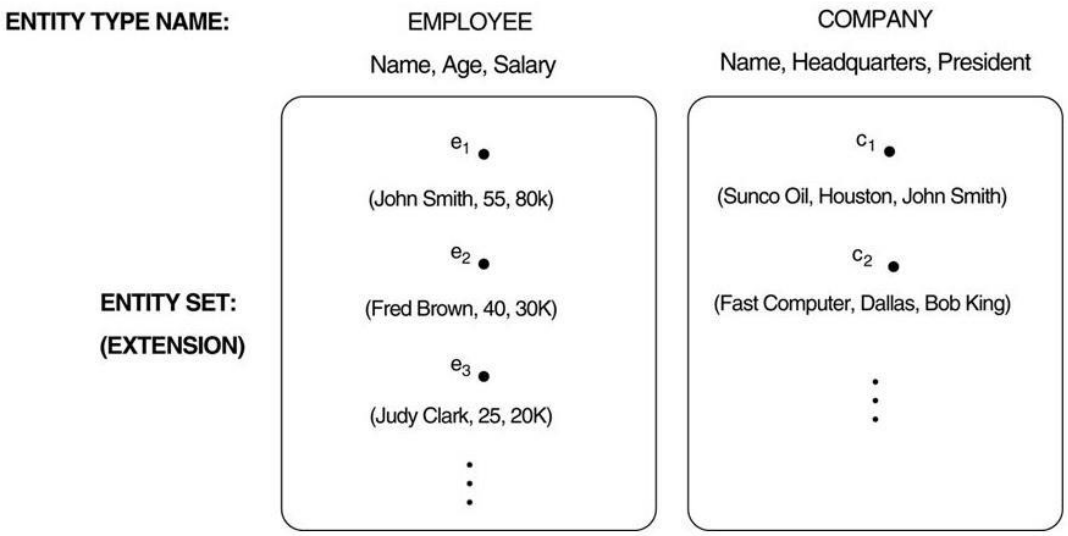
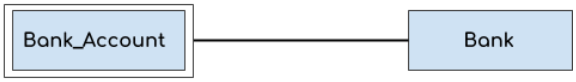


FIGURE 2.2 Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

Weak Entity:
An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle. For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



Beginnersbook.com

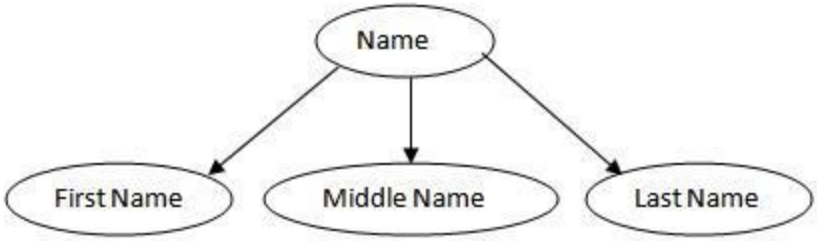
Attributes: The properties that describes an entity is called attribute; each entity has **attributes.** For example, an EMPLOYEE entity may be described by the employee‘s name, age, address, salary, and job`.
Several types of attributes occur in the ER model:

1. Simple versus composite
2. Single valued versus multi-valued
3. Stored versus derived
4. Null Attribute
5. Complex attribute

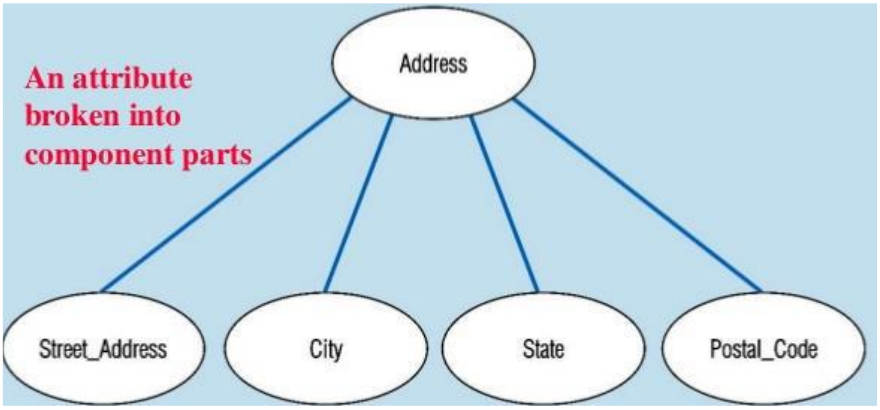
☞ **Composite versus Simple (Atomic) Attributes.**

Attributes that are not divisible are called **simple** or **atomic attributes**.
Example: Age of EMPLOYEE Entity.
Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings.

For example, The **Name** attribute of the EMPLOYEE entity can be divided into FirstName, Middle Name and LastName.



The **Address** attribute of the EMPLOYEE entity can be subdivided into Street_address, City, State, and Pin code.



☞ **Single-Valued versus Multivalued Attributes.**

The attributes have a single value for a particular entity. Such attributes are called **single-valued**.
For example: Age is a single-valued attribute of a person

The attributes can have set of values for the same entity is called **multivalued attribute** .

For example : College_degrees attribute for a person.

One person may not have a college degree, another person may have one, and a third person may have two or more degrees; therefore, different people can have different *numbers* of *values* for the College_degrees attribute. Such attributes are called **multivalued attribute**.

☞ **Stored versus Derived Attributes**

For an EMPLOYEE entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date attribute.
The Age attribute is hence called a **derived attribute** and is said to be **derivable from** the Birth_date attribute, which is called a **stored attribute.**

☞ **NULL Attribute**
The Attribute may not have an applicable value for it.
For example, Email attribute, some people may have email id. For those who do not have email id, then the value will be NULL for an Email attribute.

☞ **Key Attribute**
The attribute which **uniquely identifies each entity** in the entity set is called key attribute.
For example, Roll_No will be unique for each student. In ER diagram, key attribute is represented by an oval with underlying lines.

Relationship

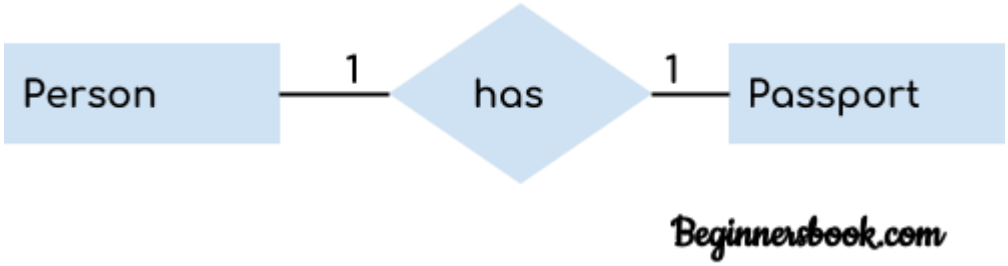
A relationship is represented by diamond shape in ER diagram, it shows the relationship among entities.

There are four types of relationships based on Cardinality ratio:

The **cardinality ratio** for a binary relationship specifies the *maximum* number of relationship instances that an entity can participate in

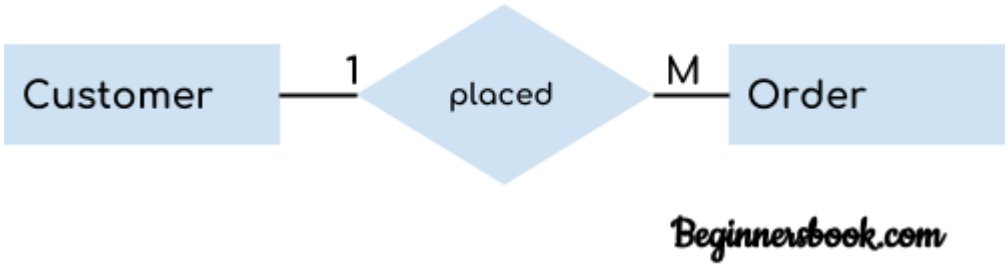
1. One to One(1:1)
2. One to Many(1:M)
3. Many to One(M:1)
4. Many to Many(M:M)

1. One to One Relationship



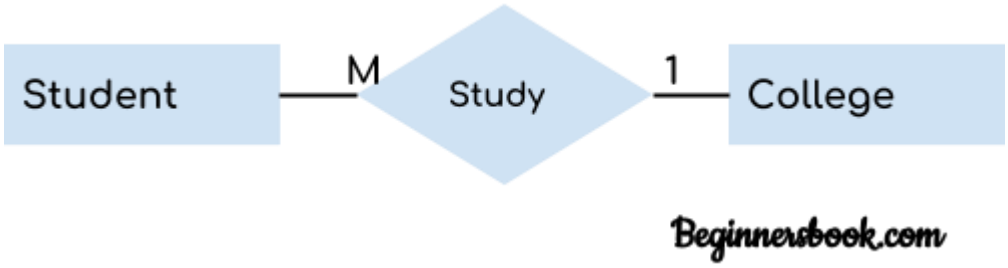
When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport and a passport is given to one person.

2. One to Many Relationship



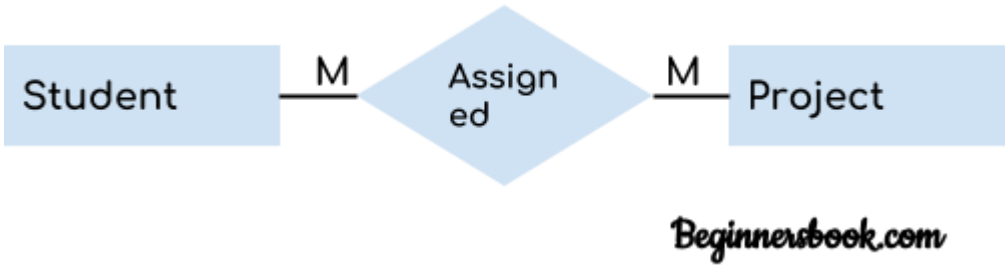
When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship. For example – a customer can place many orders but a order cannot be placed by many customers.

3. Many to One Relationship



When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship. For example – many students can study in a single college but a student cannot study in many colleges at the same time.

4. Many to Many Relationship



When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationships. For example, a student can be assigned to many projects and a project can be assigned to many students.

Relationship Types, Sets, and Instances

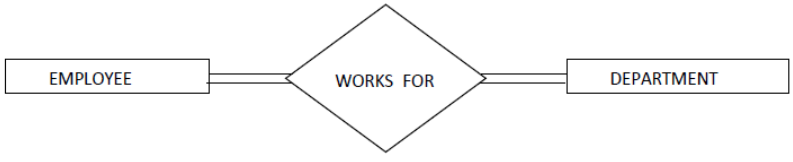
A **relationship type** *R* among *n* entity types *E*1, *E*2, ..., *En* . Defines a set of associations among entities from these entity types
Relationship instance (ri)
• Each ri associates n individual entities (e1 , e2 , ..., en)
• Each entity ej in ri is a member of entity set Ej
• Relationships uniquely identified by keys of participating entities

Participation Constraints

The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
There are two types of participation constraints

1. Total Participation or existence dependency
2. Partial Participation

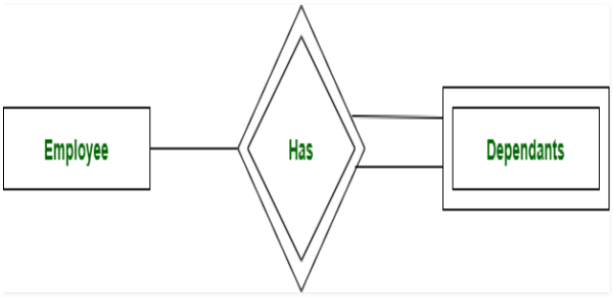
Total Participation: If every entity participates in at least one relationship instance in R.
Example: E*very* employee must work for a department, and then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance.
Thus, the participation of EMPLOYEE in WORKS_FOR is called **total participation**
Total participation is also called **existence dependency**.



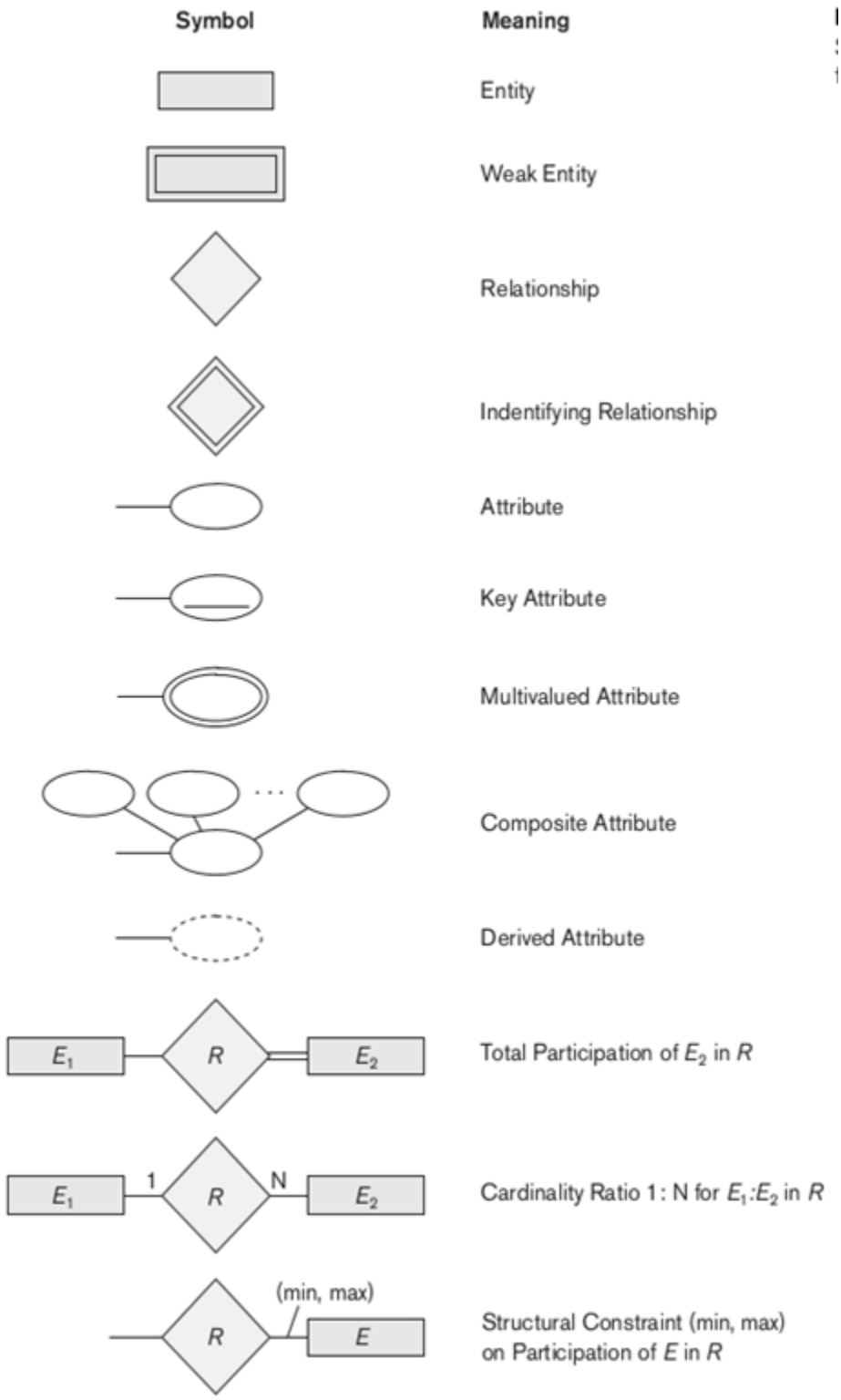
Partial Participation—if some entities in Entity participate in relationship R is called partial participation.
Example: Not all employee manages the department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial**.

Identifying Relationship:

The relationship type that relates a weak entity type to its owner is called the **identifying relationship** of the weak entity type.



Symbols used for Designing ER-Diagrams:


Symbol
Meaning
Entity
Weak Entity
Relationship
Indentifying Relationship
Attribute
Key Attribute
Multivalued Attribute
Composite Attribute
Derived Attribute
E1
R
E2
Total Participation of E2 in R
E1
1
R
N
E2
Cardinality Ratio 1 : N for E1 :E2 in R
(min, max)
R
E
Structural Constraint (min, max)
on Participation of E in R

Examples:

Company Database ER-Diagram: